

Plan de trabajo — Equipo AeroCode

Final CODEFEST AD ASTRA 2026 · del viernes 18 de septiembre (21:30) al sábado 19

Hito	Hora	Qué se evalúa
Reto 1 en línea	Sábado 08:00 a 12:30	El endpoint <code>agent.aerocode...</code> responde de forma continua. ADL lanza su batería de preguntas y de ataques de <i>prompt injection</i> .
Entrega del Reto 2	Sábado 12:30	Tablero <code>dashboard.aerocode...</code> público. Los expertos escriben sus propias preguntas.
Pitch	Según la agenda del evento	Problema, arquitectura, resultados, demostración, desafíos, fenómenos, hallazgos y verificación de fuentes.

1. Cómo se califica (para decidir prioridades)

Reto 1:

Bloque	Peso	Qué mide
Calidad	40 %	Relevancia 30 %, fidelidad al contexto recuperado 30 %, toxicidad 15 %, tono 25 %
Eficiencia	20 %	Tokens, número de llamadas y latencia, comparados con los demás equipos
Seguridad	20 %	Resistencia a <i>prompt injection</i> 75 %, análisis estático 25 %
Diseño	20 %	Ficha del agente y documento de arquitectura

Reto 2:

Bloque	Peso	Qué mide
Ejecución dinámica	55 %	Que el agente active el componente correcto con los datos correctos ante cada pregunta
Propuesta de diseño por fenómeno	40 %	Justificada en el documento de arquitectura
Calidad del código	5 %	Análisis estático

Reglas que no se pueden romper:

- Repositorio **privado**, con ADL y los evaluadores como colaboradores.
- Tres Dockerfile, uno por recurso, desplegados en Coolify.
- Tablero **solo con datos reales**, cada dato rastreable hasta `doc_id` y `chunk_id`.
- **Nada de puntajes de riesgo inventados.**
- La bolsa de modelos es de **USD 100**.

2. Estado actual (viernes 21:30)

Componente	Estado
Investigación (fenómenos, artículos, arquitectura, benchmarks)	Lista en <code>docs/investigacion/</code> y en el PDF
Agente (<code>agent/</code>): contrato §2.4, tres agentes con LangGraph, filtro de inyección, ficha del agente, Docker	Hecho, con 13 pruebas pasando y CI activo. La imagen construye y arranca en 21 s con 2,6 GB de RAM.
Latencia de la recuperación	Crítico: 25 a 29 s por pregunta en CPU. Hay que bajarla a menos de 3 s.

Componente	Estado
Cliente de modelos	Soporta el gateway OpenAI-compatible (clave <code>sk-</code>) y Bedrock. Falta la URL base del gateway.
Frontend de chat (<code>frontagent/</code>)	Hecho: Next.js con panel de evidencia y traza, lint y build limpios
Datos del tablero (<code>dashboard/datos/</code>)	En construcción: SQLite trazable, geometrías de Colombia y del mundo
API y frontend del tablero	Pendientes
Coolify	En configuración
Documento de arquitectura y pitch	Pendientes

3. Frentes y responsables

Juan · Núcleo del agente y rendimiento (Reto 1)

1. **Latencia de la recuperación a menos de 3 s:** - Perfilar qué paso consume el tiempo: encoder, disperso, reranker o selección de documentos. - Reducir candidatos del reranker, recortar el pipeline heredado (no necesitamos los 3 documentos ni los 10 fragmentos) y hacer caché de la metadata. - Medir otra vez dentro del contenedor.
2. **Modelos:** conectar la clave al gateway en cuanto se conozca la URL base. Correr la mini-evaluación de modelos (orquestador, corpus y visualización) sobre las 50 preguntas de ejemplo, con un tope de USD 5. Fijar los modelos en `.env` y en `agent_card.json`.
3. **Calidad de respuesta:** ajustar prompts por tono, relevancia y fidelidad; decidir el número de fragmentos (entre 4 y 6) según la medición.
4. **Integración final:** congelar la rama `main` del agente y coordinar el despliegue definitivo del Reto 1.

Nicolás · Plataforma, despliegue y agente de visualización (Reto 2)

1. **Coolify:** - Llave SSH en GitHub y recursos `agent`, `frontagent` y `dashboard` con build pack Dockerfile y *Base Directory* por carpeta. - Dominios, puertos y *No redirect*; variables de entorno y *healthchecks*. - Verificar que la imagen pesada del agente construye en el servidor. Si no, bajar peso: modelos ONNX o un reranker más pequeño.
2. **API del tablero (`dashboard/api`):** - Un endpoint por componente del catálogo, con filtros, que devuelva los datos y sus referencias `doc_id` y `chunk_id`. - `/api/evidencia/{chunk_id}` para el texto original. - `/api/visualizar`: instrucción en lenguaje natural → agente → especificación → datos.
3. **Ejecución dinámica (55 %):** probar con preguntas variadas que el agente elija el componente y los filtros correctos; ajustar el catálogo y el prompt del agente de visualización.
4. **Monitoreo** del despliegue durante la ventana de evaluación, con redeploy inmediato ante cualquier falla.

Esteban · Frontend del tablero (Reto 2)

1. SPA del tablero (React con TypeScript), servida por el mismo contenedor de la API.
2. Componentes: - Mapa coroplético de Colombia (alertas por departamento y municipio, capas por economía ilícita). - Mapa mundial de menciones por país. - Línea de tiempo por fenómeno, con reaparición de entidades. - Matriz de calor. - Red de entidades con expansión de vecinos. - Cuadrante intensidad frente a tendencia (solo conteos). - Barras de composición. - Panel de evidencia.
3. Filtros globales (fenómeno y fechas) y *brushing and linking* entre vistas.
4. Caja de instrucción en lenguaje natural que activa el componente pedido. Estilo coherente con `frontagent` y accesible (paleta apta para daltonismo, leyendas y unidades).

Gabriela · Documentación, diseño por fenómeno y pitch

- 1. **Documento de arquitectura** (`docs/ARQUITECTURA.md`): - Agentes, orquestación, herramientas y modelos, con la justificación de cada decisión apoyada en los artículos y benchmarks de `docs/investigacion/`. - Seguridad, eficiencia, trazabilidad y despliegue.
- 2. **Propuesta de diseño por fenómeno (40 % del Reto 2)**: para F1, F2 y F3, qué preguntas analíticas se responden, qué componente responde cada una y por qué, siguiendo las tareas analíticas del Anexo B.
- 3. **README raíz** con instrucciones de despliegue y de uso.
- 4. **Pitch**: guion y diapositivas que cubran los ocho criterios, con los subfenómenos diferenciales (`01_fenomenos/subfenomenos_wow.md`) y las tres demostraciones. Solo datos verificados y con fuente.

Santiago · Calidad, seguridad y verificación

- 1. **Batería de evaluación propia**: las 50 preguntas de ejemplo más preguntas fuera de alcance. Medir relevancia, fidelidad y tono con un juez fijo y registrar tokens, llamadas y latencia por pregunta.
- 2. **Red teaming**: 30 o más ataques de *prompt injection*, directos, ocultos en la pregunta, de cambio de rol y de exfiltración de claves. Reportar los que pasen para reforzar el filtro. Revisar que las preguntas legítimas no se bloqueen.
- 3. **Análisis estático**: ruff, bandit, eslint y dependencias vulnerables en los tres componentes; que el CI quede en verde.
- 4. **Verificación de fuentes para el pitch**: confirmar cada cifra que se vaya a decir contra su `doc_id` o su URL.
- 5. **Ensayo de la demostración** de principio a fin sobre los dominios reales.

4. Cronograma

Bloque	Juan	Nicolás	Esteban	Gabriela	Santiago
Vie 21:30 – 00:00	Perfilar y bajar la latencia	Coolify: llaves, recursos y primer despliegue del agente	Esqueleto de la SPA y del mapa de Colombia	Estructura de ARQUITECTURA.md y propuesta por fenómeno	Batería de preguntas y ataques
Sáb 00:00 – 03:00	Gateway, modelos y mini-evaluación	API del tablero y <code>/api/visualizar</code>	Línea de tiempo, red, matriz y evidencia	Redacción de la arquitectura con las decisiones de Juan y Nicolás	Correr la batería contra el agente desplegado; reportar
Sáb 03:00 – 06:00	Ajuste de prompts y calidad	Pruebas de ejecución dinámica	Filtros globales, <i>brushing</i> y pulido	README y guion del pitch	Análisis estático y seguridad; verificación de cifras
Sáb 06:00 – 08:00	Congelar el Reto 1 y hacer el despliegue final	Verificar dominios, <i>healthchecks</i> y logs	Integración con la API desplegada	Diapositivas	Ensayo de punta a punta del Reto 1
Sáb 08:00 – 12:30	Monitorear el Reto 1; solo correcciones críticas	Cerrar y desplegar el Reto 2; monitorear	Pulido final del tablero	Ensayo del pitch	Ensayo del Reto 2 con preguntas nuevas

Reglas de trabajo:

- `git pull --rebase` antes de cada push.
- Commits pequeños en español.
- Nadie sube secretos: las claves van solo en las variables de entorno de Coolify.
- Después de las 06:00, cualquier cambio en `agent/` pasa por el responsable del núcleo del agente.

5. Riesgos y planes B

Riesgo	Plan B
La recuperación sigue lenta	Quitar el reranker (usar solo la fusión densa y dispersa) o usar uno más pequeño; bajar a 4 fragmentos
La imagen del agente no construye en Coolify (tamaño o tiempo)	Descargar los modelos al arrancar, con volumen persistente, o usar variantes ONNX más livianas
No aparece la URL del gateway	Pedirla a ADL de inmediato; mientras tanto, probar con Bedrock nativo si la clave lo permite
El gateway es lento o gasta de más	Mantener el tope de tokens y cambiar el modelo por variables de entorno, sin reconstruir la imagen
El tablero no llega completo	Priorizar mapa, línea de tiempo, red y evidencia, que cubren las cuatro tareas del Anexo B
Falla un despliegue durante la evaluación	Dejar identificado el último commit estable y redeployarlo desde Coolify

6. Definición de terminado

- ☐ POST `https://agent.aerocode.codefest2026.augusta.avaldigitallabs.com/chat` responde el contrato de la §2.4 en menos de 10 s.
- ☐ `frontagent.aerocode...` permite chatear, ver citas y ver la traza.
- ☐ `dashboard.aerocode...` activa componentes con instrucciones en lenguaje natural, todo trazable a `doc_id` y `chunk_id`.
- ☐ `agent_card.json` coincide con los modelos realmente usados.
- ☐ `docs/ARQUITECTURA.md` y el README están completos, con la propuesta por fenómeno justificada.
- ☐ CI en verde y ningún secreto en el repositorio.
- ☐ ADL y los evaluadores están invitados como colaboradores del repositorio privado.
- ☐ El pitch está ensayado dentro del tiempo y con las fuentes verificadas.